


```
def forward(self, preds, labels):
    loss = 0
    for i in range(preds.shape[0]):
        sum_x = torch.sum(preds[i])
        sum_y = torch.sum(labels[i])
        sum_xy = torch.sum(preds[i]*labels[i])
        sum_x2 = torch.sum(torch.pow(preds[i],2))
        sum_y2 = torch.sum(torch.pow(labels[i],2))
        N = preds.shape[1]
        pearson = (N*sum_xy - sum_x*sum_y)/(torch.sqrt((N*sum_x2 -
torch.pow(sum_x,2))*(N*sum_y2 - torch.pow(sum_y,2))))
        loss += 1 - pearson

    loss = loss/preds.shape[0]
    return loss
```

□ □ □ □ 가 □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

[illegible][illegible][illegible]

[illegible]

☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

□ □ □ □ □ □ □ □ □ □

☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

□ □ □ □ □ □ □ □ □ □ □ □

[illegible]

```
@lru_cache(maxsize=32)
def _detrrend_projection(signal_length, lambda_value):
    signal_length = int(signal_length)
    lambda_value = float(lambda_value)
    h = np.identity(signal_length)
    ones = np.ones(signal_length)
    minus_twos = -2 * np.ones(signal_length)
    diags_data = np.array([ones, minus_twos, ones])
    diags_index = np.array([0, 1, 2])
    d_mat = spdiags(diags_data, diags_index, (signal_length - 2), signal_length).toarray()
    return h - np.linalg.inv(h + (lambda_value**2) * np.dot(d_mat.T, d_mat))

def cached_detrrend(input_signal, lambda_value):
    input_signal = np.asarray(input_signal)
    projection = _detrrend_projection(input_signal.shape[0], float(lambda_value))
    return np.dot(projection, input_signal)
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □

[illegible]

☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

가
가

